



DFRWS USA 2025 - Selected Papers from the 25th Annual Digital Forensics Research Conference USA

If at first you don't succeed, try, try again: Correcting TLSH scalability claims for large-dataset malware forensics

Jordi Gonzalez

The MITRE Corporation, 7525 Colshire Drive, McLean, 22102, VA, USA

ARTICLE INFO

Keywords:

Locality-sensitive hashing
Malware analysis

ABSTRACT

Malware analysts use Trend Micro Locality-Sensitive Hashing (TLSH) for malware similarity computation, nearest-neighbor search, and related tasks like clustering and family classification. Although TLSH scales better than many alternatives, technical limitations have limited its application to larger datasets. Using the Lean 4 proof assistant, I formalized bounds on the properties of TLSH most relevant to its scalability and identified flaws in prior TLSH nearest-neighbor search algorithms. I leveraged these formal results to design correct acceleration structures for TLSH nearest-neighbor queries. On typical analyst workloads, these structures performed one to two orders of magnitude faster than the prior state-of-the-art, allowing analysts to use datasets at least an order of magnitude larger than what was previously feasible with the same computational resources. I make all code and data publicly available.

1. Introduction

The growing volume of both malicious and benign software presents a growing burden to malware analysts and security vendors. They must accurately identify connections between malware samples while avoiding false associations between innocuous files and malware, and between unrelated malware families.

Locality-sensitive hashing (LSH) (Indyk and Motwani, 1998; Haq and Caballero, 2021) helps analysts solve this problem by providing a precise (Oliver et al., 2013) dimensionality reduction technique, whereby more similar pieces of software have higher spatial proximity. This capability enables at-scale approximate nearest-neighbor searches (Breitinger et al., 2014) and, in turn, clustering (Oliver et al., 2021; Bak et al., 2020), antivirus whitelisting (Smart Whitelisting Using Locality Sensitive, 2017), detection (Intelligence, 2021; Naik et al., 2019a), malware campaign tracking (Naik et al., 2019b), and threat information sharing (Almahmoud et al., 2022; Jordan et al.).

Trend Micro Locality-Sensitive Hashing (Oliver et al., 2013; Oliver, 2024a) (TLSH) emerged as a standard locality-sensitive hash function for malware analysis. However, efficiently searching for similar hashes

in large TLSH datasets remains a challenge: even though it is possible to compute TLSH hashes rapidly for a set of inputs, and even though it is possible to compare different pairs of hashes rapidly, finding nearest-neighbors is computationally demanding and scales poorly with corpora size. The root of this issue is in TLSH's distance function, which violates the triangle inequality and, therefore, limits the use of metric data structures for nearest-neighbor searches. No correct, non-approximate, publicly available algorithm was found that addressed this gap. This work makes several contributions toward closing it.

First, this work uses the Lean 4 theorem prover (de Moura and Ullrich, 2021) to provide formal, tight bounds on the triangle inequality violations for the TLSH distance function and its various sub-components. This exposes an error in prior research that underestimated the bounds by a factor of over 20 (see Table 1).

Second, based on those theoretical results, this work presents two TLSH-specific nearest-neighbor acceleration structures: one, based on tries (Fredkin, 1960), and another, based on vantage-point (VP) trees (Yianilos, 1993; Uhlmann, 1991).

Third, this work evaluates the performance of these structures on

E-mail address: jgonzalez@mitre.org.

<https://doi.org/10.1016/j.fsidi.2025.301922>

Available online 1 August 2025

2666-2817/© 2025 The Author(s). Published by Elsevier Ltd on behalf of DFRWS. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Table 1
TLSH triangle inequality distance violations.

Component	Violation
Header Distance Violations	
Checksum-distance	0
L-distance	22
q ₁ -distance	12
q ₂ -distance	12
Total Header Distance	46
Body Distance Violations	
Per-bucket body distance	3
Total Body Distance (128 buckets)	384
Overall Violation	430

real-world and synthetic data, demonstrating >10x throughput on common workloads compared to corrected versions of the prior state-of-the-art.

Finally, all code is made available at <https://github.com/mitre/fast-search-for-tlsh>.

2. Background

2.1. Design of TLSH

The TLSH whitepaper (Oliver et al., 2013) accurately describes TLSH behavior. This subsection summarizes the aspects of the whitepaper most relevant to this work.

TLSH maps arbitrary byte streams to fixed-length hashes. The hashes are split into a “header” component and a “body” component. As illustrated in Fig. 1, these two components have several subcomponents, or “features.” The precise semantics of these features are unimportant to this work, but their layout is illustrated by Fig. 1.

Computing the distance between two samples using TLSH is a three-step process:

First, TLSH hashes are computed for both files. Second, each feature in one TLSH hash is compared against the corresponding feature in the other TLSH hash, and the feature distances are recorded. Finally, these distances are summed to produce the final TLSH distance. Fig. 2 presents this visually.

The formulae used to compute feature distances are specific to the features being compared. For example, the formula for checksum (the first feature in Fig. 1) distance can only output distances of zero or one.

Not mentioned in the whitepaper, but deeply relevant to this work, is that most of these formulae are not metrics in the mathematical sense because they do not all obey the triangle inequality. However, they are semi-metrics because they are all symmetric, non-negative, and only yield distances of zero when two features are the same.

The proofs of these claims are omitted for reasons of triviality: for

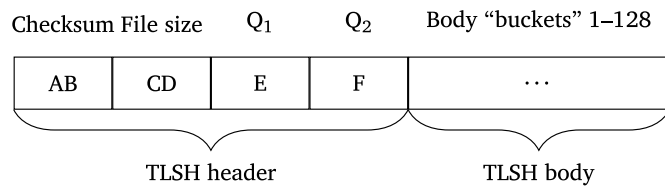


Fig. 1. Structure of a TLSH hash.

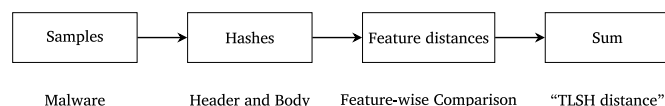


Fig. 2. TLSH file comparison pipeline.

example, all TLSH formulae accumulate distance by either adding modular distances (a counting-based distance metric (Oliver et al., 2013)), positive constants, or absolute values of expressions (Oliver et al., 2013). As these all constitute natural numbers, and as the naturals are closed under addition, TLSH distances must also be naturals; and proof that TLSH distance violates the triangle inequality follows trivially from the proof that TLSH can violate the triangle inequality by 430 distance units, which I include in Appendix A, complete with constructive examples. Others have also made constructive (if not maximal) proofs available (Baggett, 2023).

2.1.1. TLSH for malware forensics

TLSH offers several practical attributes for malware forensics and analysis: TLSH is open and permissively licensed (Oliver, 2024a), so unlike proprietary LSH schemes, there are no limitations or costs associated with its access or usage. TLSH also performs competitively in comparative evaluations, particularly in terms of accuracy and robustness to adversarial attacks (Oliver and Hagen, 2021). Finally, as a corollary of doing well amongst high-uptake LSH schemes, TLSH benefits from strong network effects, having been adopted by platforms like VirusTotal.

These features are of value to analysts as they facilitate the sharing and broader use of TLSH hashes. For example, because VirusTotal adopted TLSH, analysts can conduct TLSH-similarity queries on the entire VirusTotal corpus (VirusTotal, 2024).

2.1.2. TLSH limitation

Unfortunately, a TLSH technical limitation disrupts the viability of large-corpora TLSH nearest-neighbor queries. Indeed, “due to performance reasons”, VirusTotal throttles TLSH queries to a rate of 15 per minute for paying users and prohibits their use entirely for non-paying users. Moreover, VirusTotal’s TLSH indexing and querying algorithms are private, and a review of the literature found no public alternatives. This leaves no straightforward way for those in the industry to bear the cost of self-hosting a similar service. The lack of such tooling necessarily constrains analysts’ ability to use TLSH with large malware datasets.

This limitation stems from the fact that TLSH is only a semi-metric, not a metric: it violates the triangle inequality.

In a metric space, the triangle inequality suggests that if Alice and Bob are close, and if Bob and Eve are close, then one can use their closeness to bound the distance between Alice and Eve. If TLSH were a metric, VP trees (Yianilos, 1993) could enable efficient nearest-neighbor searches, because TLSH hashes could be laid out in such a way that these bounds can direct a search, and, in turn, enable pruning of large areas of the search space.

Because TLSH does not obey the triangle inequality, the use of metric-based techniques for nearest-neighbor searches is limited. Analysts ostensibly must either conduct *exhaustive linear scans* of the entirety of a dataset, every time that a query is conducted; or analysts must use *approximate nearest-neighbor search* and sacrifice precision, recall, or both.

2.2. Related work

2.2.1. Research into TLSH scalability

The official TLSH documentation contradicts the claims of this paper, noting that “TLSH is very fast at nearest-neighbor search at scale [...] being a distance metric (as per the mathematical definition) and hence has logarithmic search times [...] and in particular [obeys] the triangle inequality” (Oliver, 2021a). However, this is misleading, as the authors now publicly acknowledge that “the [TLSH] distance function does not obey the triangle inequality” (Oliver, 2024b) and is only “an approximate distance metric” (Oliver et al., 2020).

Prior work to improve TLSH’s scalability has had varying degrees of applicability. For instance, Trend Micro Research documented one improvement to the use of TLSH for clustering, centered around the

parallelization of a clustering algorithm (Ali et al., 2020). While this can improve throughput on a system in (at best) direct proportion to the amount of available system parallelism, it does not solve the high computational costs of TLSH queries themselves; rather, it just allows that cost to be distributed across more CPU cores.

Other Trend Micro Research work presented two different acceleration structures for TLSH nearest-neighbor search: a random forest and a VP tree (Oliver et al., 2021). This work avoids the former because it only gives approximate results, as noted in the prior work. The latter, as was also noted, is a path toward an exact algorithm, making it highly relevant. This work builds off their VP tree concept.

A careful reader may note that VP trees were previously implied to be incompatible with non-metric distance functions. The prior work attempts to address this based on the premise that because TLSH is still “within a constant of a metric” (Oliver et al., 2020), and because “distance functions that are within a constant [factor] of a metric” can still be accelerated, a special VP tree can still work (Oliver et al., 2020, 2021) if it considers additional nodes. The flaw with that work is in the implementation: the authors assume that the constant factor is 20 (Oliver, 2021b), which is incorrect. To be correct, the authors should have used 430 (see Table 1). Unfortunately, applying this correction reduces the performance of their implementation to that of unaccelerated, linear scanning (see Fig. 3).

2.2.2. Non-TLSH alternatives

Several mainstream LSH schemes, like ssdeep (Kornblum, 2006) and sdhash (Roussev, 2010), can be used as TLSH alternatives; however, these do not solve the underlying scalability problem and, in comparative evaluations, often fail to outperform TLSH (Oliver et al., 2013; Pagani et al., 2018; Azab et al., 2014).

Other LSH schemes map to proper metric spaces (Gu et al., 2013; Oprea et al., 2014), theoretically enabling efficient nearest-neighbor search. In spite of this advantage, and in spite of other potential advantages held by competing algorithms, they haven’t yet displaced TLSH; accordingly, TLSH still enjoys an ecosystem size advantage.

Within digital malware forensics specifically, several LSH schemes exist as specialized algorithms, like PermHash for Chromium extensions (Wilson, 2023), or peHash for PE files (Wicherski, 2009). These specializations can be highly constraining: peHash, for example, will never identify relationships in code similarity between Mac, Windows, and Linux payloads, as ELF files are not PE files, and neither are Mach-O files.

While many non-LSH techniques exist (Haq and Caballero, 2021), none were identified in the literature with the compactness (Oliver et al., 2013), throughput (Oliver et al., 2013; Haq and Caballero, 2021; Li et al., 2019), generality (Oliver et al., 2013), and wide uptake of LSH schemes.

3. Methods

This research is divided into three sections: theoretical analysis to formalize TLSH’s triangle inequality violations, algorithm design to exploit the theoretical results, and comparison of the devised algorithms to alternatives.

3.1. Theory

I first proved bounds on the degree to which TLSH subcomponents could violate the triangle inequality, formalizing the proof using the Lean v4.14.0 (de Moura and Ullrich, 2021) proof-assistant and mathematical library (The mathlib community, 2020).

Specifically, letting H_1, H_2, H_3 be any three distinct TLSH hashes, and letting $d(x, y)$ represent the contribution of a feature of TLSH to the total TLSH distance, I solved for the tightest bounds on c in the triangle inequality, $d(H_1, H_3) \leq d(H_1, H_2) + d(H_2, H_3) + c$.

Building on those results, I showed bounds for the TLSH distance function and its constituent header and body components. For the proof

itself, see Appendix A.

3.2. Algorithm design

I implemented and tested the following algorithms. For each, Appendix B includes pseudocode.

3.2.1. VP tree

I implemented a VP tree (Yianilos, 1993; Uhlmann, 1991), inspired by prior work (Oliver et al., 2020).

Unlike existing methods, this VP tree exclusively indexes the TLSH header component, which, in theory, confers two advantages over indexing the entirety of a TLSH hash: faster VP tree index construction because body feature distance never needs to be computed; and faster VP tree queries because header component distance violates the triangle equality to a much lesser degree than body component distance, which allows for more aggressive pruning during searches.

Because body component distance still matters, it is checked before a candidate is added to the resulting nearest-neighbor list; if the sum of header and body distance for two hashes is outside the “cutoff” for what defines a nearby-neighbor, it is pruned.

3.2.2. Trie with schema-learning

I implemented a novel, trie-based (Fredkin, 1960) approach to nearest-neighbor queries. Rather than search for nearby neighbors in the order that TLSH features are laid out in a hash, it uses a greedy, randomized algorithm to find an ordering, or “schema,” that maximizes the number of nodes pruned at shallow levels of the search tree.

Trie search follows the ordering laid out in the schema. Helpfully, schemas can be saved and reused on different datasets. In trie search, the proofs of each TLSH feature’s contributions to the total triangle inequality violation are used to constrain the search radius.

3.2.3. Replication of prior work

I implemented and evaluated the VP tree design described by Trend Micro Research (Oliver, 2021b) in Rust in order to evaluate performance relative to prior work.

Concerning the correctness issue outlined in Section 2.2.1, I tested “corrected” and “uncorrected” versions of their algorithm.

3.2.4. Linear scanning

Linear scanning served as a control.

Though largely outside the scope of the paper, the source code for this work includes a parallel, Tensorflow-accelerated (Abadi et al., 2015) Python library for working with TLSH on large datasets. Appendix C covers the performance of its linear scanning routine.

3.3. Empirical evaluation

3.3.1. Datasets

I evaluated performance on two datasets: randomly generated synthetic TLSH hashes, and a convenience sample of 1,263,016 TLSH hashes sourced from the VirusTotal metadata for a local repository of VirusShare (VirusShare.com, 2024) data.

The latter dataset represents the entirety of an employer-maintained collection of data. Data was not filtered prior to or during the evaluation. Furthermore, and in the interest of transparency, the data is provided in the source code associated with the paper (see Appendix A).

3.3.2. Workloads

This work examined three groups of nearest-neighbor search workloads, based on common analytical tasks.

“Near” neighbors were defined as those with a TLSH distance of at most 30, as this is a standard industry and academic choice (Hutelmeyer and Borre, 2024; Joyce et al., 2023) with TLSH. Larger cutoffs trade precision for recall (Oliver et al., 2013).

1. All-to-all workloads:

An analyst may receive a set of samples to triage and want to cluster them.

To approximate inefficient clustering algorithms, I do pairwise comparisons of every sample in corpora of sizes 5000 and 10,000.

2. Fixed-query-size, variable-corpora-size workloads:

An analyst may receive a set of samples to triage and want to check which are novel.

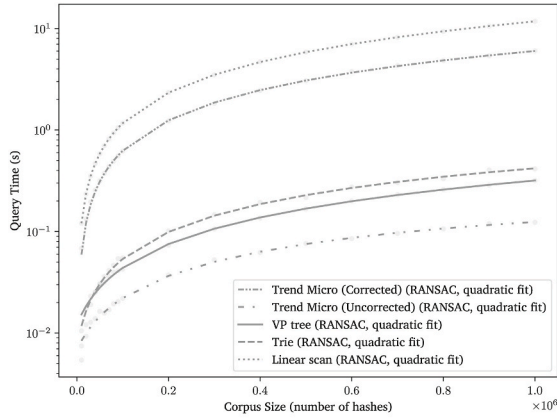
To approximate this task, I measured the time it took to conduct 1000 queries on various-sized subsets of a larger corpus. The queries were chosen randomly from the larger corpus rather than its subset.

3. Variable-query-size, variable-corpora-size workloads:

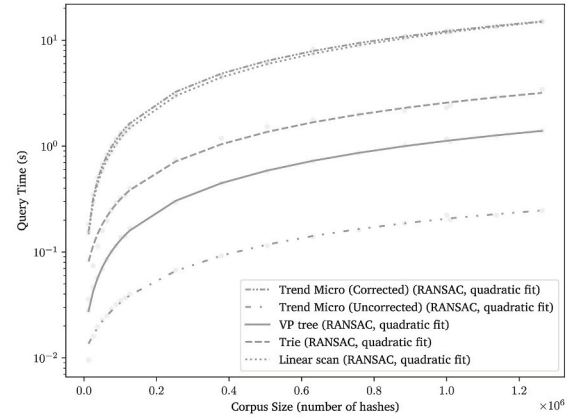
An analyst may use TLSH and a specialized clustering algorithm requiring very few comparisons.

To approximate more efficient clustering algorithms, I measured the time it took to query a random 10 % of corpora of various sizes. The sampling procedure is the same as for the fixed-query-size, variable-corpora-size workload.

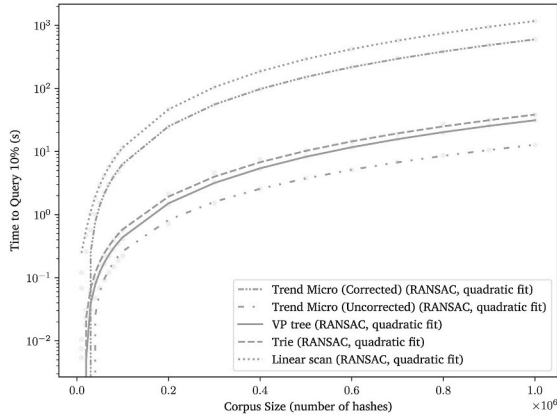
To assess whether the choice of 30 as a distance cutoff biased the results and to capture alternative workloads, I evaluated cutoffs from 1 to 1000. Note that at cutoffs above 200, TLSH may exceed a false-positive rate of 50 % (Oliver et al., 2013), so measurements past that



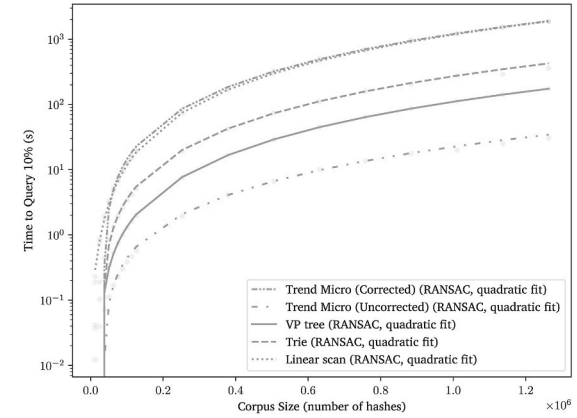
(a) Relationship between corpus size and time to query 1,000 hashes in randomly generated hash corpora.



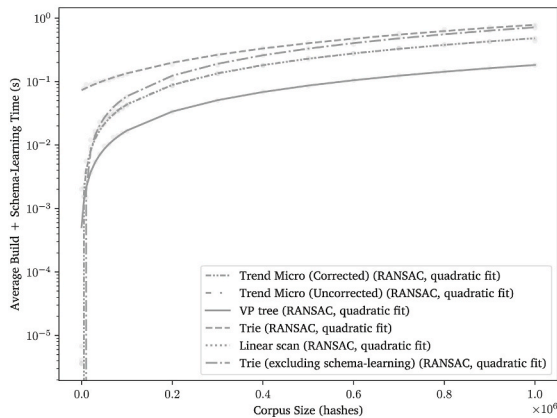
(b) Relationship between corpus size and time to query 1,000 hashes in VirusShare-derived hash corpora.



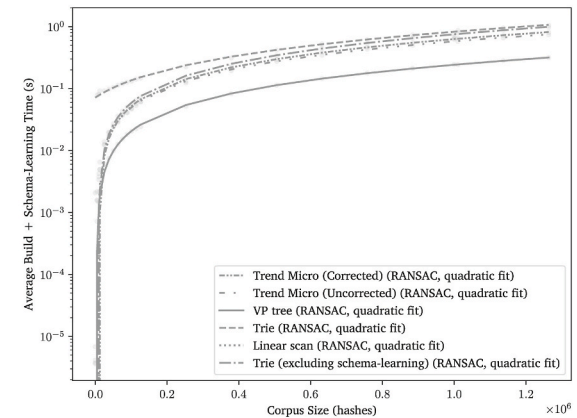
(c) Relationship between corpus size and total build and query time, when querying 10% of randomly generated hash corpora.



(d) Relationship between corpus size and total build and query time, when querying 10% of VirusShare-derived hash corpora.



(e) Relationship between corpus size and index build time for randomly generated hashes.



(f) Relationship between corpus size and index build time for VirusShare-derived hashes.

Fig. 3. Assorted performance metrics for the various algorithms with cutoff of 30.

point are of questionable utility.

To assess whether aspects of these workloads were “shifted” from query-time to preprocessing-time in a way that might prove too computationally demanding or wasteful for specific tasks, I also evaluated the time spent building the acceleration structures.

Note that because a single schema may be reused with different corpora, the choice to do schema-learning (rather than reuse a single schema) represents a trade-off between data structure efficiency and preprocessing time. Although this has little influence on results, I show measurements of trie preprocessing with and without schema-learning.

3.3.3. Benchmark environment

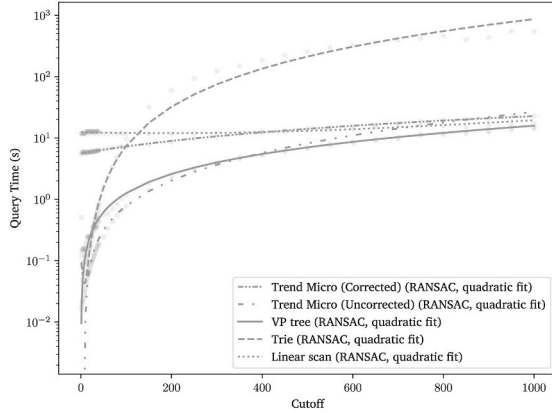
I ran all benchmarks 11 times on a 12-core M2 Max MacBook Pro

using high-performance mode on macOS 15.2. Reported results represent median execution times. The testing harness and associated algorithms were built with Rust 1.83.0. The benchmark suite recorded CPU clock speeds using the sysinfo crate (Sysinfo, 2024). The records showed no indications of throttling.

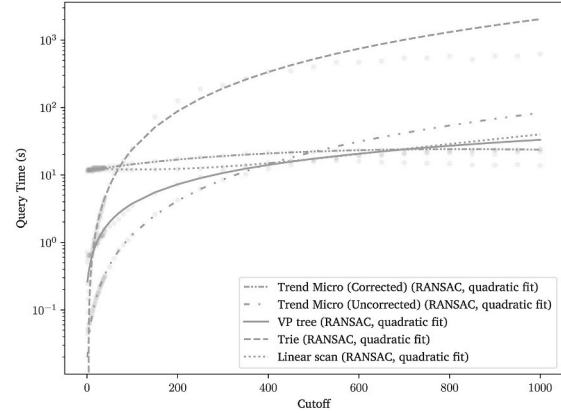
4. Results

4.1. Theory

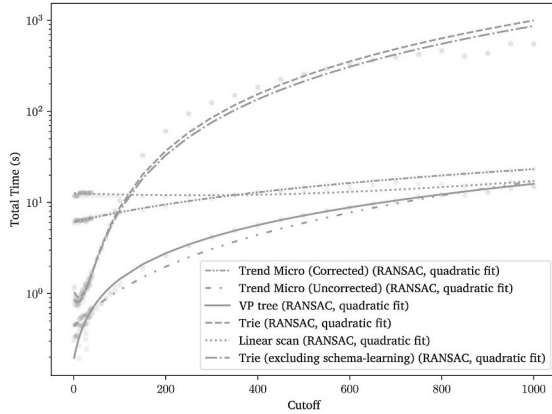
I quantified the contribution of different TLSH features to the total violation of the triangle inequality. For information regarding the proof, please see [Appendix A](#).



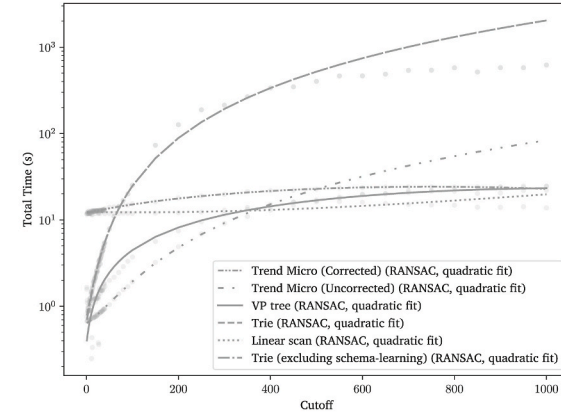
(a) Relationship between cutoff and time to query 1,000 of 1,000,000 random hashes.



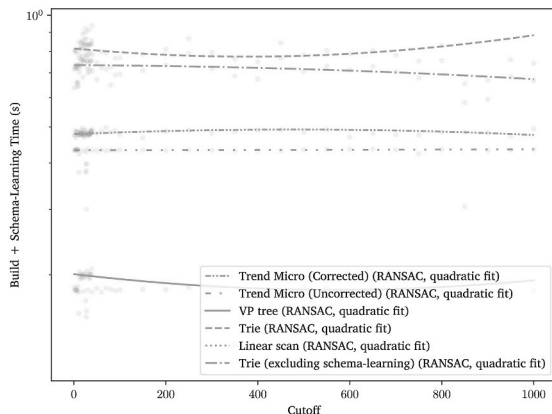
(b) Relationship between cutoff and time to query 1,000 of 1,000,000 VirusShare-derived hashes.



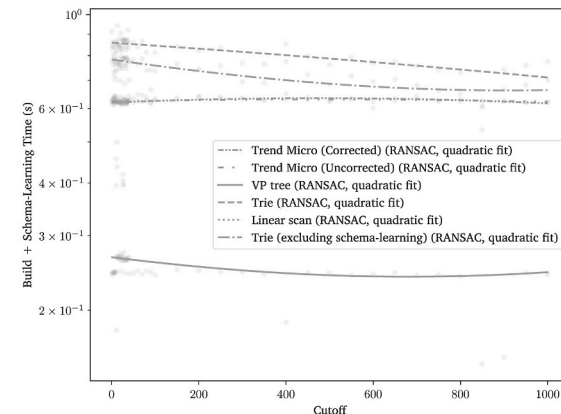
(c) Relationship between cutoff and total build and query time for 1,000 queries of 1,000,000 random hashes.



(d) Relationship between cutoff and total build and query time for 1,000 queries of 1,000,000 VirusShare-derived hashes.



(e) Relationship between cutoff and build time (with and without schema-learning) for 1,000,000 random hashes.



(f) Relationship between cutoff and build time (with and without schema-learning) for 1,000,000 VirusShare-derived hashes.

Fig. 4. Effect of different cutoff levels on algorithm performance.

Note the precise semantics of these features is of low direct relevance to this work.

4.2. Scaling by workload and data source

Fig. 3 shows algorithm performance when the cutoff for similarity is 30 and corpora sizes are varied. Fig. 4 shows the performance of the algorithms when only the cutoff for similarity is varied.

4.3. Fixed-workload results

Both indices outperformed linear scanning on the 10,000-to-10,000, cutoff-of-30, clustering-like workloads. The VP tree was strictly faster than the trie on real-world data.

Results were similar on 1000-to-1,000,000 workloads.

5. Discussion

The results make for several substantial contributions to the TLSH scalability literature.

The analysis of TLSH's triangle inequality violations was the most relevant to prior literature because it demonstrated errors in prior work. Table 1 showed that the triangle inequality could be violated by a constant factor of 430, far exceeding the prior estimate of 20 (Oliver, 2021b). After correcting this error in the prior work, its performance degraded to that of linear scanning (see Fig. 3b).

The results on synthetic data were much better for the acceleration structures than the results on real-world data. This discrepancy likely stems from the tighter distribution of TLSH features in real-world data. For example, file length—a key TLSH feature—has much lower variance in real-world datasets than in randomly generated data. This tighter clustering means hashes have more plausibly nearby neighbors, reducing opportunities for pruning during queries. This analysis focuses on real-world data to avoid drawing biased conclusions.

5.1. VP tree

Compared to the corrected prior work and linear scanning, I see much better performance—over an order of magnitude greater throughput at the standard cutoff of 30—with the presented VP tree. The uplift was in both index construction (Fig. 3f) and querying (Fig. 3d), due to reduced computational overhead during construction (as body distance does not get used) and more aggressive pruning during searches, respectively.

The VP tree demonstrated performance advantages during index construction at all cutoff levels and against all tested data structures (see Fig. 4f).

Although the VP tree underperformed linear scanning at extremely wide similarity cutoffs, at every cutoff below 200—when ≥ 50 % false positive rates are known to occur (Oliver et al., 2013) and broader cutoffs are likely to be impractical—the VP tree was the fastest technique (see Fig. 4d).

The advantage was particularly pronounced in Fig. 4b, which represents cases where index construction times amortize away; here, the VP tree was the best performing up to cutoffs of approximately 400.

5.2. Trie

I diverged from prior work by introducing a novel, trie-based VP tree alternative for TLSH nearest-neighbor search. At very strict cutoffs, the trie outperforms the VP tree and the prior work by an order of magnitude, but this advantage quickly vanishes at higher cutoff values. At cutoffs above 10, the VP tree shows a consistent performance advantage, and at cutoffs above 100, the trie underperforms linear scanning. This stark regression is likely because, with each query, the trie performs an exhaustive breadth-first search. It is only because of aggressive pruning,

which requires tight cutoffs, that trie search is performant.

5.3. Linear scanning

Because linear scanning performance is unaffected by cutoff (see Fig. 4d), linear scanning may be advantageous in certain ultra-high-cutoff use cases. On smaller corpora, even when linear scanning is slower than querying indexed structures, highly demanding workloads like all-to-all queries can still be completed in seconds (see Fig. 5). Consequently, there may be cases where linear scanning is preferable to indexed searches for speed or convenience.

Nevertheless, linear scanning performance degrades quickly with larger corpora and more queries. Fig. 6 demonstrates this limitation, showing a large performance gap between the index-based algorithms and linear scanning.

5.4. Implications

The results were highly pronounced on clustering workloads, where the best algorithm—the VP tree—delivered a $10 \times$ performance uplift compared to the state-of-the-art (see Fig. 3b). Though difficult to see on a log scale, performance *scaling* was also much better with the VP tree

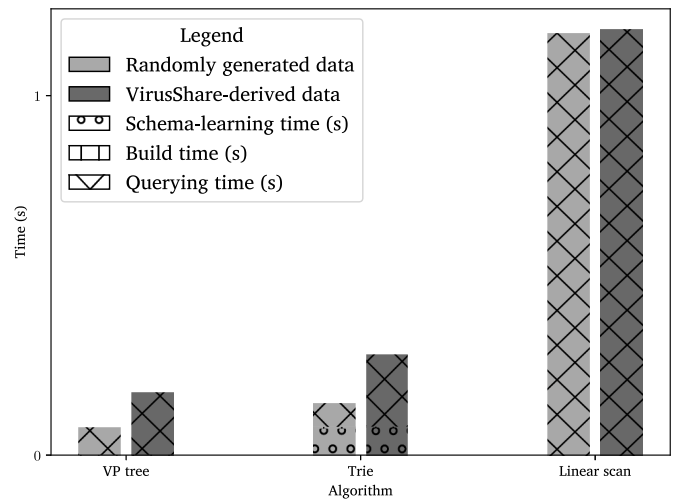


Fig. 5. Median profile of an all-to-all workload involving a 10,000-hash corpus, by data source.

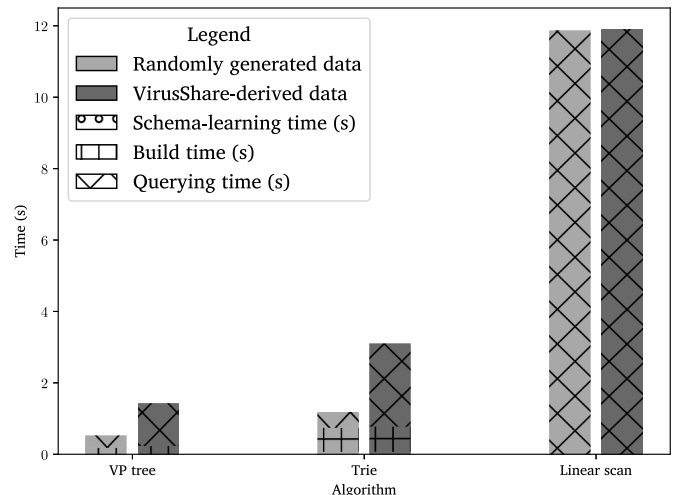


Fig. 6. Median profile of a 1000-to-1,000,000 workload, by data source.

than with the prior work.

Unlike clustering workloads, where the number of nearest-neighbor queries grows with the size of the dataset, malware corpus search APIs like VirusTotal’s “Advanced Corpus Search” (VirusTotal, 2024) represent workloads that consist entirely of a single nearest-neighbor query. For these APIs and workloads, performance improvements for queries directly translate to increased API or analyst capacity: at a cutoff of 30, compared to the prior state-of-the-art, a nearest-neighbor search API using the VP tree can either dispatch ten times as many queries or query a dataset ten times the size, with the same compute budget.

6. Conclusion

I’ve shown, using a formal proof assistant, fundamental limitations in TLSH and their impact on prior TLSH nearest-neighbor search implementations. Building on the formal results, I’ve presented two data structures that could overcome these limitations: one based on a vantage-point tree, and another based on a trie-like structure.

Experimental results show that on real-world data, for nearest-neighbor queries and associated clustering workloads, these data structures provide one to two orders of magnitude greater throughput relative to the state of the art. Except at the most stringent cutoffs for what constitutes a “near neighbor,” the vantage-point tree was the fastest of the two new data structures. These performance improvements

allow analysts to process datasets an order of magnitude larger than what was previously feasible with the same resources.

It is hoped that—given the large design space for this particular problem—the formal results, benchmarking tools, and code for working with TLSH may serve as a foundation for further improvements.

Acknowledgments

I want to acknowledge Corvus Forensics for maintaining the Virus-Share dataset, which made it viable to test on real-world data. I would also like to thank MITRE and MITRE’s sponsors for sponsoring this research, particularly Brian Shaw, Christopher Andersen, Dan Perret, Dr. Justin Brunelle, Frank Posluszny, Laurence Hunter, Morgan Keiser, and Tim McNevin.

I want to give additional thanks to Brian Shaw, Christopher Andersen, Dr. Justin Brunelle, and Tim McNevin for their invaluable constructive criticism; Frank Posluszny, for his feedback and for providing me with the malware metadata from which the real-world TLSH digests were extracted; and Laurence Hunter, for his mentorship, generosity, and feedback, without which this could not have been written.

This software (or technical data) was produced for the U.S. Government and is subject to the Rights in Data-General Clause 52.227–14, Alt. IV (May 2014) – Alternative IV (Dec 2007).

Appendix A. Supplemental Material

All supplemental material can be found at <https://github.com/mitre/fast-search-for-tlsh>.

Appendix B. Pseudocode

All algorithms take in a data structure holding a corpus, a query, and a radius used as the cutoff for similarity search.

The VP tree is constructed as any other: assume every node in the tree is a vantage-point with radius of size `node.threshold`. Little improvement was observed with different vantage-point selection strategies. The approach described as “prior literature” differs only in 430 being used as `MAX_HEADER_VIOLATION`, and with the total distance being used instead of header distance.

Algorithm 1. Query logic for VP tree

```

function RANGE_QUERY(tree, query, radius)
  results ← [ ]
  stack ← [tree.root]
  new_radius ← radius + MAX_HEADER_VIOLATION
  while stack not empty do
    node ← stack.pop
    header_dist ← HEADER_DIST(node.point, query)
    body_dist ← BODY_DIST(node.point, query)
    if header_dist + body_dist ≤ radius then
      append node.point to results
    end if
    if header_dist - new_radius ≤ node.threshold then
      append node.left to stack
    end if
    if header_dist + new_radius ≥ node.threshold then
      append node.right to stack
    end if
  end while
  return results
end function

```

The trie has both a searching component and a schema-learning component.

Algorithm 2. Trie schema-learning logic

```

function LEARN_SCHEMA(data, cutoff)
  sample_n ← MIN(64, |data|)
  sampled_data ← randomly sample sample_n from data
  schema ← [ ]
  max_cutoffs ← 0
  loop
    best_feature ← null
    best_num_cutoffs ← max_cutoffs
    best_feature_by_sum ← null
    best_sum ← 0
    feature_range ← 0.36 ▷ 36 = # TLSH features
    for each i in feature_range do
      if schema contains i then
        continue
      end if
      trial ← schema + [i]
      num_cutoffs ← 0
      total_sum ← 0
      for each v in sampled_data do
        pair_cutoffs ← 0
        pair_sum ← 0
        for each u in sampled_data do
          diff ← FEATURE_DIST(u, v, trial)
          if diff ≥ cutoff then
            pair_cutoffs ← pair_cutoffs + 1
          end if
          pair_sum ← pair_sum + diff
        end for
        num_cutoffs ← num_cutoffs + pair_cutoffs
        total_sum ← total_sum + pair_sum
      end for
      if total_sum > best_sum then
        best_sum ← total_sum
        best_feature_by_sum ← i
      end if
      if num_cutoffs > best_num_cutoffs then
        best_num_cutoffs ← num_cutoffs
        best_feature ← i
      end if
    end for
    if best_feature not null then
      append best_feature to schema
    else if best_feature_by_sum not null then
      append best_feature_by_sum to schema
    else
      break
    end if
    max_cutoffs ← best_num_cutoffs
  end loop
  return schema
end function

```

Algorithm 3. Trie query logic

```

function TRIE_SEARCH(node, query, radius, schema)
  if node is a leaf then
    return FILTER(node.points, query, radius, schema) ▷ Filter out would-be false-positives using remaining features
  end if
  results ← [ ]
  feature ← first feature in schema
  value ← GET_FEATURE_VALUE(query, feature)
  max_diff ← COMPUTE_MAX_DIFF(feature, radius)
  for all possible candidate value s.t. candidate diff ∈ [0, max_diff] do ▷ diff > max_diff → dist > radius
    dist ← FEATURE_DIFF(value, candidate, feature)
    if dist ≤ radius then
      child ← node[candidate]
      radius_budget ← radius - dist
      schema' ← TAIL(schema)
      sub_results ← TRIE_SEARCH(child, query, radius_budget, schema')
    end if
  end for
  return results
end function

```

(continued on next page)

(continued)

```

for each sub_result in sub_results do append sub_result to results
end for
end if
end for
return results
end function

```

For completeness, `linear_scan` is as follows:

Algorithm 4. Linear scan

```

function LINEAR_SCAN(corpus, query, radius)
  results ← [ ]
  for digest in corpus do
    header_dist ← HEADER_DIST(digest, query)
    body_dist ← BODY_DIST(digest, query)
    if header_dist + body_dist ≤ radius then
      append digest to results
    end if
  end for
  return results
end function

```

Appendix C. Tensorflow-accelerated scanner

I authored a Tensorflow-accelerated linear scanning routine to facilitate using TLSH on large datasets, within Python notebooks. I benchmarked it against a linear scanning routine powered by `py-tlsh` (Py-tlsh, 2024), the official C++ Python extension for fast TLSH operations.

The benchmarks used Rust 1.83.0, Python 3.12.7, Tensorflow 2.18.0 (Abadi et al., 2015), numpy 2.0.1 (Harris et al., 2020), and `py-tlsh` 4.7.2 (Py-tlsh, 2024).

I included Rust linear scanner performance for illustrative purposes only, as the Rust and Python benchmark harnesses differ, limiting result comparability.

The benchmark results were as follows:

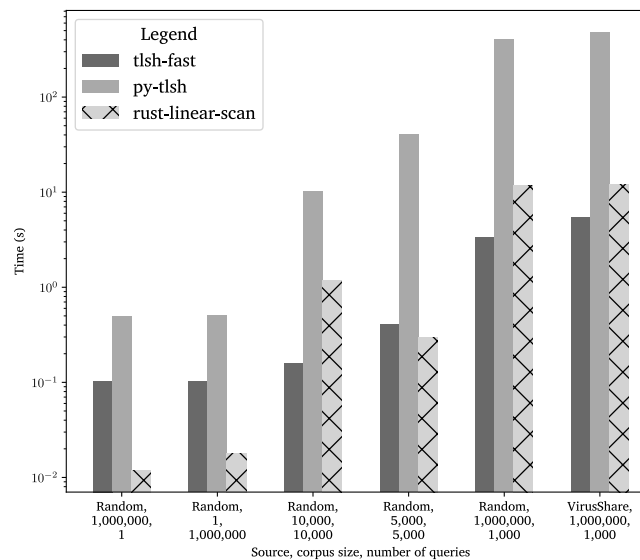


Fig. C.7. Performance of linear-scan implementations.

The Tensorflow-enhanced Python library outperformed the `py-tlsh`-based scanner across all measured workloads.

I attribute a roughly 10x performance uplift, relative to the `py-tlsh`-based scanner, to parallelism; and the residual performance uplifts to two factors: that Tensorflow optimizes memory access patterns, particularly on all-to-all tasks; and that Tensorflow batches queries, which reduces the time spent in the Python interpreter.

The benchmark code is available in [Appendix A](#).

References

- Abadi, Martín, Agarwal, Ashish, Barham, Paul, Brevdo, Eugene, Chen, Zhifeng, Craig, Citro, Corrado, Greg S., Davis, Andy, Dean, Jeffrey, Devin, Matthieu, Ghemawat, Sanjay, Goodfellow, Ian, Harp, Andrew, Irving, Geoffrey, Isard, Michael, Jia, Y., Jozefowicz, Rafal, Kaiser, Lukasz, Kudlur, Manjunath, Levenberg, Josh, Mané, Dandelion, Monga, Rajat, Moore, Sherry, Murray, Derek, Olah, Chris, Schuster, Mike, Shlens, Jonathon, Steiner, Benoit, Sutskever, Ilya, Talwar, Kunal, Tucker, Paul, Vincent, Vanhoucke, Vasudevan, Vijay, Viégas, Fernanda, Vinyals, Oriol, Warden, Pete, Wattenberg, Martin, Wicke, Martin, Yuan, Yu, Zheng, Xiaoqiang, 2015. TensorFlow: large-scale machine learning on heterogeneous systems. <https://www.tensorflow.org/>.
- Ali, M., Hagen, J., Oliver, J., 2020. Scalable malware clustering using multi-stage tree parallelization. In: 2020 IEEE International Conference on Intelligence and Security Informatics (ISI). IEEE, Arlington, VA, USA, pp. 1–6. <https://doi.org/10.1109/ISI49825.2020.9280546>.
- Almahmoud, A., Damiani, E., Otrok, H., 2022. Hash-comb: a hierarchical distance-preserving multi-hash data representation for collaborative analytics. *IEEE Access* 10, 34393–34403. <https://doi.org/10.1109/ACCESS.2022.3158934>.
- Azab, A., Layton, R., Alazab, M., Oliver, J., 2014. Mining malware to detect variants. In: 2014 Fifth Cybercrime and Trustworthy Computing Conference, pp. 44–53. <https://doi.org/10.1109/CTC.2014.11>.
- Baggett, D., 2023. TLSH distance metric appears to violate triangle inequality. <https://github.com/trendmicro/tlsh/issues/130#issue-1623514292>.
- Bak, M., Papp, D., Tamás, C., Buttyán, L., 2020. Clustering IoT malware based on binary similarity. In: NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium, pp. 1–6. <https://doi.org/10.1109/NOMS47738.2020.9110432>.
- Breitinger, F., White, D., Guttman, B., McCarrin, M., Roussev, V., 2014. Approximate matching: definition and terminology. In: Tech. Rep. NIST Special Publication (SP) 800-168. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-168>. Jul.
- de Moura, L., Ullrich, S., 2021. The lean 4 theorem prover and programming language. In: Platzer, A., Sutcliffe, G. (Eds.), *Automated Deduction – CADE 28*. Springer International Publishing, Cham, pp. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.
- Fredkin, E., 1960. Trie memory, commun. *ACM* 3 (9), 490–499. <https://doi.org/10.1145/367390.367400>.
- Gu, X., Zhang, Y., Zhang, L., Zhang, D., Li, J., 2013. An improved method of locality sensitive hashing for indexing large-scale and high-dimensional features. *Signal Process.* 93 (8), 2244–2255. <https://doi.org/10.1016/j.sigpro.2012.07.014>.
- Hag, I.U., Caballero, J., 2021. A survey of binary code similarity. *ACM Comput. Surv.* 54 (3), 51:1–51:38. <https://doi.org/10.1145/3446371>.
- Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., Oliphant, T.E., 2020. Array programming with NumPy. *Nature* 585 (7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.
- Hutelmeyer, P., Borre, R., 2024. Implementing TLSH based detection to identify malware variants. Dec. https://tech.target.com/blog/implementing_TLSH_based_detection.
- Indyk, P., Motwani, R., 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In: *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing, STOC '98*. Association for Computing Machinery, New York, NY, USA, pp. 604–613. <https://doi.org/10.1145/276698.276876>.
- Intelligence, M.T., 2021. Combing through the fuzz: using fuzzy hashing and deep learning to counter malware detection evasion techniques. Jul. <https://www.microsoft.com/en-us/security/blog/2021/07/27/combing-through-the-fuzz-using-fuzzy-hashing-and-deep-learning-to-counter-malware-detection-evasion-techniques/>.
- B. Jordan, R. Piazza, T. Darley, Stix version 2.1, Tech. rep., OASIS Standard, latest stage: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>. URL <https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html>.
- Joyce, R.J., Patel, T., Nicholas, C., Raff, E., 2023. AVScan2Vec: feature learning on antivirus scan data for production-scale malware corpora. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec '23*. Association for Computing Machinery, New York, NY, USA, pp. 185–196. <https://doi.org/10.1145/3605764.3623907>.
- Kornblum, J., 2006. Identifying Almost Identical Files Using Context Triggered Piecewise Hashing. <https://doi.org/10.1016/j.diin.2006.06.015>. Sep.
- Li, Y., Jang, J., Ou, X., 2019. Topology-aware hashing for effective control flow graph similarity analysis. In: Chen, S., Choo, K.-K.R., Fu, X., Lou, W., Mohaisen, A. (Eds.), *Security and Privacy in Communication Networks*. Springer International Publishing, Cham, pp. 278–298. https://doi.org/10.1007/978-3-030-37228-6_14.
- Naik, N., Jenkins, P., Savage, N., 2019a. A ransomware detection method using fuzzy hashing for mitigating the risk of occlusion of information systems. In: 2019 International Symposium on Systems Engineering (ISSE), pp. 1–6. <https://doi.org/10.1109/ISSE46696.2019.8984540>.
- Naik, N., Jenkins, P., Savage, N., Yang, L., 2019b. Cyberthreat hunting - Part 2: tracking ransomware threat actors using fuzzy hashing and fuzzy C-means clustering. In: 2019 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE). IEEE, New Orleans, LA, USA, pp. 1–6. <https://doi.org/10.1109/FUZZ-IEEE.2019.8858825>.
- Oliver, J., 2021a. TLSH - technical overview. Apr. <https://tlsh.org/papers.html>.
- Oliver, J., 2021b. Tlsh/tlshCluster/pylib/hac lib.py. Sep. <https://github.com/trendmicro/tlsh/blob/96536e3f5b9b322b44ce88d36126121685e45a77/tlshCluster/pylib/hac.lib.py#L143>.
- Oliver, J., 2024a. GitHub - trendmicro/tlsh. <https://github.com/trendmicro/tlsh>.
- Oliver, J., 2024b. TLSH distance metric appears to violate triangle inequality. Jan. <https://github.com/trendmicro/tlsh/issues/130#issuecomment-1906886178>.
- Oliver, J., Hagen, J., 2021. Designing the elements of a fuzzy hashing scheme. In: 2021 IEEE 19th International Conference on Embedded and Ubiquitous Computing (EUC). IEEE, Shenyang, China, pp. 1–6. <https://doi.org/10.1109/EUC53437.2021.00028>.
- Oliver, J., Cheng, C., Chen, Y., 2013. TLSH – a locality sensitive hash. In: 2013 Fourth Cybercrime and Trustworthy Computing Workshop, pp. 7–13. <https://doi.org/10.1109/CTC.2013.9>.
- Oliver, J., Ali, M., Hagen, J., 2020. HAC-T and fast search for similarity in security. In: 2020 International Conference on Omni-Layer Intelligent Systems (COINS), pp. 1–7. <https://doi.org/10.1109/COINS49042.2020.9191381>.
- Oliver, J., Ali, M., Liu, H., Hagen, J., 2021. Fast clustering of high dimensional data clustering the malware bazaar dataset. https://tlsh.org/papersDir/n21_opt_cluster.pdf.
- Oprisa, C., Checicheş, M., Năndrean, A., 2014. Locality-sensitive hashing optimizations for fast malware clustering. In: 2014 IEEE 10th International Conference on Intelligent Computer Communication and Processing (ICCP), pp. 97–104. <https://doi.org/10.1109/ICCP.2014.6936960>.
- Pagani, F., Dell'Amico, M., Balzarotti, D., 2018. Beyond precision and recall: understanding uses (and misuses) of similarity hashes in binary analysis. In: *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy, CODASPY '18*. Association for Computing Machinery, New York, NY, USA, pp. 354–365. <https://doi.org/10.1145/3176258.3176306>.
- Py-tlsh, 2024. TLSH (C++ Python extension). Sep. <https://github.com/trendmicro/tlsh>.
- Roussev, V., 2010. Data fingerprinting with similarity digests. In: Chow, K.-P., Sheno, S. (Eds.), *Advances in Digital Forensics VI*. Springer, Berlin, Heidelberg, pp. 207–226. https://doi.org/10.1007/978-3-642-15506-2_15.
- Smart whitelisting using locality sensitive hashing. Mar. https://www.trendmicro.com/en_us/research/17/c/smart-whitelisting-using-locality-sensitive-hashing.html.
- Sysinfo, 2024. crates.io: Rust package registry. Dec. <https://crates.io/crates/sysinfo>.
- The mathlib community, 2020. The lean mathematical library. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020*, pp. 367–381. <https://doi.org/10.1145/3372885.3373824>. New Orleans, LA, USA, January 20–21, 2020.
- Uhlmann, J.K., 1991. Satisfying general proximity/similarity queries with metric trees, *Information Processing Letters*, 40 (4), 175–179. [https://doi.org/10.1016/0020-0190\(91\)90074-R](https://doi.org/10.1016/0020-0190(91)90074-R).
- VirusShare.com. Sep. <https://virusshare.com/>.
- VirusTotal, 2024. Advanced corpus search. Jul. <https://docs.virustotal.com/reference/in-telligence-search>.
- Wicherski, G., 2009. peHash: a novel approach to fast malware clustering. In: *USENIX Workshop on Large-Scale Exploits and Emergent Threats*. <https://www.semanticscholar.org/paper/peHashApproach-to-Fast-Malware-Clustering-Wicherski/a52ddc15377bc9f2ef1f237afa41d324f321bb9b>.
- Wilson, Jared, 2023. Perlmash — No curls necessary. May. <https://cloud.google.com/blog/topics/threat-intelligence/perlmash-no-curls-necessary>.
- Yianilos, P.N., 1993. Data structures and algorithms for nearest neighbor search in general metric spaces. In: *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '93*. Society for Industrial and Applied Mathematics, USA, pp. 311–321.